# Auditoría de IA con McCabe y cobertura

## Sistema de citas de medicina familiar

Desarrollo del taller · Pruebas de caja blanca

**Andrés Quisilema**

Universidad Internacional del Ecuador
Ingeniería en Sistemas de la Información
19 de octubre de 2026

Código ejecutable, grafo de control, tres caminos independientes, cobertura, oráculos y mutaciones.

# Propósito y alcance

Desarrollé un sistema local de citas de medicina familiar con Node.js, Express, SQLite y Alpine.js. Después audité su lógica con un grafo de flujo de control, cálculo de McCabe, pruebas con resultados exactos y mutaciones. La entrega sigue los tres pasos del taller y responde el cuestionario final del PDF.

El programa permite reservar, listar, buscar y cancelar citas. El servidor valida las cuatro reglas críticas: edad, horarios, choques y penalización tardía. La interfaz usa blanco como superficie principal y azul marino oscuro en títulos y acciones. Se aplicaron las guías de diseño de los dos repositorios indicados, con el alcance de verificación declarado al final.

# Reglas adoptadas

El PDF no establece edades límite, horarios concretos, duración ni monto. Se fijaron como supuestos del ejercicio: pediatría de 0 a 17 años, adultos desde los 18 hasta los 120; lunes a viernes 08:00–12:00 y 13:00–17:00; citas de 30 minutos; zona Quito (UTC−05:00). Una cita cancelada libera su horario. Los intervalos son [inicio, fin), por lo que una consulta puede empezar justo cuando termina la anterior.

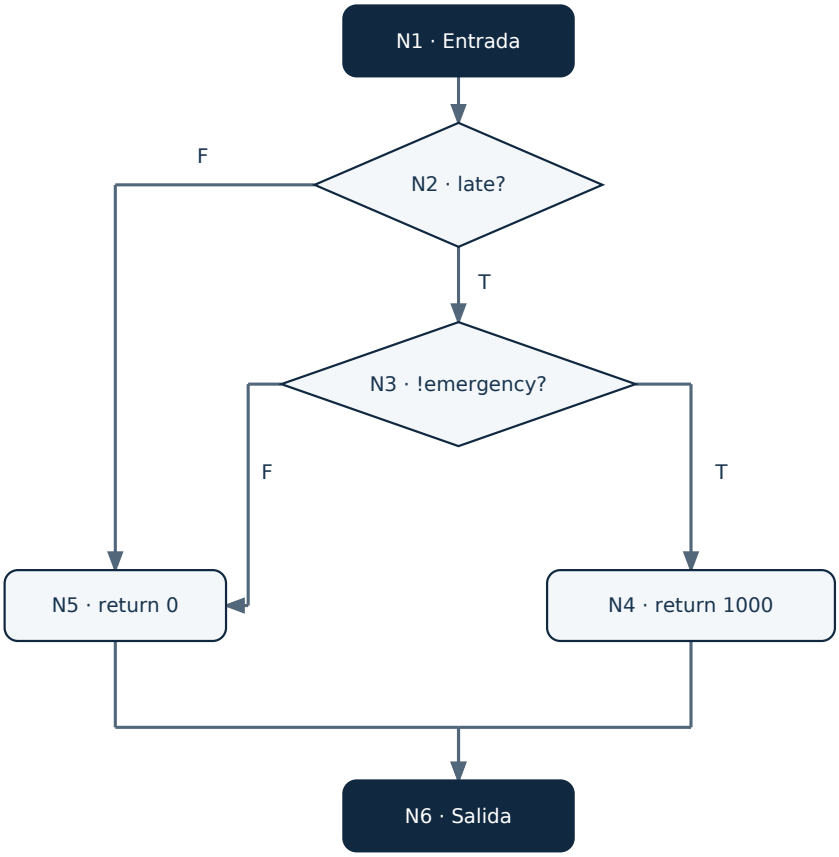
Una cancelación es tardía si faltan menos de 24 horas. Con 24 horas exactas o más no genera cargo. Si es tardía y no existe emergencia, registra 1000 centavos (USD 10); una emergencia exime el cargo. Se rechaza cancelar una cita iniciada. La demostración registra el cargo sin procesar pagos.

# Prompt maestro de desarrollo

“Genera un sistema de citas de médico familiar con Node.js (Express), SQLite y Alpine.js. Implementa reservas, listado y cancelaciones. Valida edad entera de 0 a 120: pediatría hasta los 17 y adultos desde los 18. Atiende de lunes a viernes, 08:00–12:00 y 13:00–17:00, en Quito; citas de 30 minutos con inicio a las :00 o :30 y solo en el futuro. Impide solapamientos del mismo profesional mediante transacción e índice de SQLite. Al cancelar antes del inicio, cobra exactamente 1000 centavos si faltan menos de 24 horas y no hay emergencia. Con emergencia o con 24 horas o más, cobra cero. Conserva las canceladas en el historial. Usa errores claros y consultas parametrizadas. Diseña en blanco y azul marino oscuro siguiendo Impeccable y Emil Kowalski. Entrega código ejecutable, pruebas Jest, evidencias reproducibles y supuestos explícitos. No declares resultados sin ejecutar las comprobaciones.”

# Paso 1. Cálculo manual de la penalización

CFG de la expresión por cortocircuito. Cada arista representa una transferencia de control; T y F indican verdadero y falso.




La decisión auditada es A && B, donde A significa cancelación tardía y B significa !emergencia. En JavaScript existe cortocircuito: si A es falsa, B no se evalúa. Por eso el CFG descompone la expresión en dos nodos predicado.

```
function penalty(late, emergency) {
  if (late && !emergency) {
    return 1000;
  }
  return 0;
}
```

Nodos: N1 entrada, N2 late?, N3 !emergency?, N4 return 1000, N5 return 0 y N6 salida común. Aristas: N1→N2, N2(F)→N5, N2(T)→N3, N3(T)→N4, N3(F)→N5, N4→N6 y N5→N6.

Hay N=6 nodos, E=7 aristas y P=2 predicados. Para este grafo conectado: V(G)=E−N+2=7−6+2=3. También V(G)=P+1=2+1=3. En su representación plana aparecen dos regiones internas más la exterior: tres regiones. La letra P aquí designa predicados, no componentes conexos de la fórmula general.

| Camino | Tardía | Emergencia | Recorrido | Oráculo exacto |
|---|---|---|---|---|
| P1 | Falso | Falso | N1,N2,N5,N6 | 0 centavos |
| P2 | Verdadero | Verdadero | N1,N2,N3,N5,N6 | 0 centavos |
| P3 | Verdadero | Falso | N1,N2,N3,N4,N6 | 1000 centavos |

Estos recorridos forman una base de tres caminos independientes. V(G)=3 se refiere a esta función, no a toda la aplicación. No demuestra ausencia de defectos ni sustituye pruebas de datos, fronteras, estado o integración (Wallace et al., 1996).

# Condiciones y MC/DC

Las pruebas P1 y P3 mantienen emergencia=false y cambian late: la decisión pasa de falsa a verdadera. Las pruebas P2 y P3 mantienen late=true y cambian emergencia: la decisión también cambia. Así se justifica el efecto independiente de ambas condiciones para esta expresión. En P1, B tiene valor lógico verdadero por la entrada, pero no se ejecuta debido al cortocircuito. Esta justificación es manual; Jest/V8 no emite un certificado MC/DC. Cobertura de condiciones y MC/DC no son sinónimos: MC/DC exige demostrar la independencia (NASA, 2022).

# Paso 2. El engaño del falso 100 %

Se ejecutó una suite separada que llama penalty(true,false) y penalty(false,false) sin ninguna aserción. Recorre los retornos de 1000 y 0, pero no comprueba cuál recibe. Jest/V8 reportó 100 % en sentencias, ramas, funciones y líneas para penalty.cjs. Eso describe los contadores del proveedor, no la solidez de la auditoría.

La misma suite dejó pasar cinco mutaciones: AND por OR, emergencia invertida, cargo de 999, retorno exento de 1000 y tardanza negada. Ninguna prueba falló. El resultado reproduce el problema de los tests zombis: ejecutar una función no basta para detectar un resultado incorrecto. Las aserciones exactas son el oráculo que compara la salida con la regla del negocio.

En términos de PIE y su extensión con revelación, el test debe alcanzar el defecto, permitir que altere el estado, propagar el error hasta una salida observable y revelarlo con el oráculo. La cobertura solo aporta evidencia de ejecución; un contador no comprueba por sí mismo infección, propagación ni revelación. El ejemplo usa mutaciones para hacer observable esta diferencia.

# Paso 3. Re-prompting del auditor

“Actúa como ingeniero de QA. Audita la función penalty(late, emergency). Su CFG por cortocircuito tiene V(G)=3. Genera exactamente tres pruebas unitarias Jest para sus tres caminos independientes: (false,false)→0, (true,true)→0 y (true,false)→1000. Usa expect(...).toBe(...) para verificar el importe exacto en centavos. Relaciona cada prueba con su recorrido y demuestra manualmente los pares MC/DC. Ejecuta Jest con coverageProvider v8 y declara qué archivos mide. No confundas cobertura con corrección. En suites separadas, añade fronteras de las 24 horas y pruebas de integración de las otras reglas. Introduce mutaciones controladas y verifica que fallen por aserciones. Conserva los registros reales y declara cualquier limitación.”

La suite base contiene exactamente tres tests, uno por camino, en tests/penalty.test.cjs. Las pruebas adicionales no alteran ese requisito: pertenecen a otras suites y cubren riesgos que el número de McCabe de una sola función no representa.

# Ejecución y resultados

Entorno: Node.js 24.19.0, Express 5.1.0, Alpine.js 3.15.8 y Jest 30.2.0, en Linux. SQLite se usa mediante node:sqlite. Fecha de entrega: 19 de octubre de 2026. Dependencias fijadas en package-lock.json. Las pruebas inyectan un reloj fijo para no depender del día real de ejecución.

| Evidencia | Resultado observado | Archivo |
|---|---|---|
| Suite estricta | 82 de 82 pruebas; 4 suites aprobadas | evidence/tests.txt |
| Penalización | 100 % de las 4 métricas V8 | evidence/coverage/coverage-summary.json |
| Backend medido | 100 % sentencias, ramas y líneas; 88,88 % funciones | evidence/coverage/index.html |
| Test sin oráculos | Pasa con 100 % de métricas V8 en penalty.cjs | evidence/naive-tests.txt |
| Mutaciones manuales | 5 detectadas por suite estricta; 5 sobreviven con suite ingenua | evidence/mutations.json |
| Complejidad | penalty=3; máximo backend=9 | evidence/complexity.json |

La cobertura incluye app.cjs, db.cjs, rules.cjs y penalty.cjs. Excluye server.cjs, public/, scripts y dependencias. El reloj por defecto de createApp no se ejecuta porque las pruebas lo sustituyen; por eso la cobertura de funciones no es 100 %. Las pruebas frontend verifican el modelo de estado con Node VM y no forman parte de ese porcentaje.

# Fronteras, integración y mutaciones

Las pruebas de edad incluyen 0, 17, 18, 120, negativos, decimales y 121. Las de horario incluyen 08:00, 11:30, 12:00, 12:30, 13:00, 16:30, 17:00, fines de semana, fechas inexistentes y citas pasadas. Las cancelaciones comprueban 24 h + 1 ms, 24 h, 24 h − 1 ms y el instante de inicio. Los oráculos verifican importes, códigos HTTP y estado persistido.

Las pruebas API comprueban reserva, listado, solapamiento parcial, citas contiguas, profesionales distintos a la misma hora, doble petición concurrente, liberación del espacio al cancelar, cancelación repetida y conservación del cargo. Una transacción BEGIN IMMEDIATE mantiene unidas la búsqueda de conflicto y la inserción. Un índice único parcial refuerza el rechazo del mismo profesional/inicio activo. La prueba concurrente usa una instancia del servidor; no demuestra comportamiento bajo carga multiproceso.

| Mutación | Cambio deliberado | Suite estricta | Suite ingenua |
|---|---|---|---|
| M1 | AND por OR | Detectada | Sobrevive |
| M2 | Eliminar negación de emergencia | Detectada | Sobrevive |
| M3 | 1000 por 999 centavos | Detectada | Sobrevive |
| M4 | Exención de 0 a 1000 | Detectada | Sobrevive |
| M5 | Negar condición tardía | Detectada | Sobrevive |

El resultado es 5/5 para este conjunto manual de mutantes no equivalentes. No es una puntuación exhaustiva de toda la aplicación. El script no modifica el código original: crea copias temporales, ejecuta la suite y elimina los mutantes.

# Control humano y criterio de refactorización

El control empieza al fijar las reglas y los oráculos antes de aceptar las pruebas. Después se inspecciona el flujo, se calcula V(G), se ejecutan casos y se contrastan las salidas. La IA ayuda a escribir; el ingeniero conserva la decisión sobre supuestos, alcance y aceptación.

El analizador incluido cuenta decisiones, operadores de cortocircuito y catch; mide por separado las funciones anidadas y excluye expresiones SQL y frontend. Bajo esta convención, checkPatient tiene V(G)=9, checkSchedule=8, parseStart=5, cancellationFee=3 y penalty=3. Ninguna función analizada supera 10. Se separaron validación de paciente, calendario y penalización para mantener responsabilidades acotadas. El umbral de 10 se adopta como criterio del taller, no como ley universal de calidad.

# Cuestionario de auditoría

1. ¿Conozco V(G) antes de leer los tests? Sí. Construí el CFG de la penalización y obtuve tres caminos mediante dos fórmulas. Después comprobé que cada prueba base recorriera uno de ellos.

2. ¿El 100 % incluye oráculos fuertes? El porcentaje no permite saberlo. La suite estricta sí verifica valores exactos con toBe; la suite ingenua alcanza 100 % sin aserciones y deja pasar los cinco defectos introducidos.

3. ¿Se evaluaron todas las condiciones o solo líneas? Para la penalización se cubren los tres recorridos y se justifican los dos pares de independencia. Además, se prueban límites de edad, horario y cancelación. No atribuyo MC/DC global a toda la aplicación.

4. ¿Forcé fallos intencionales? Sí. Los cinco mutantes de la función de penalización hacen fallar aserciones en la suite estricta. La suite ingenua no detecta ninguno. Los resultados constan en evidence/mutations.json.

5. ¿La complejidad exige refactorizar? Con el criterio del taller, no hay ninguna función backend analizada por encima de 10; la máxima es 9. Si una función creciera sobre ese umbral, separaría responsabilidades y repetiría pruebas. Refactorizar no elimina la necesidad de probar.

# Diseño, límites y reproducción

Se aplicaron Impeccable y Emil Kowalski: jerarquía sobria, una tipografía autoalojada, espacio entre grupos, blanco dominante, títulos azul marino, acciones claras, foco visible y movimiento breve solo al pulsar. Se evita movimiento en acciones de teclado y con reduced-motion. La revisión estática encontró una etiqueta de 11 px y se corrigió a 12 px. Los commits consultados se registran en docs/design-direction.md.

No se contó con control-browser: queda pendiente la revisión visual del frontend y la interacción real con Alpine. Las pruebas de recursos HTTP y del modelo de estado no sustituyen esa revisión. La aplicación es local y académica: no tiene autenticación, no procesa pagos y usa datos ficticios. No se presenta como software clínico listo para producción. El PDF sí fue renderizado e inspeccionado para verificar su presentación.

Para reproducir: npm ci; npm test; npm run test:coverage; npm run test:naive; npm run test:mutations; npm run audit:complexity. Para abrir el sistema: npm start y http://localhost:3000. README.md detalla los pasos y la comprobación manual pendiente.

# Referencias

Material docente. (s. f.). Auditoría de IA con McCabe y cobertura [Presentación facilitada para la actividad].

Wallace, D. R., Watson, A. H., & McCabe, T. J. (1996). Structured testing: A testing methodology using the cyclomatic complexity metric (NIST Special Publication 500-235). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.500-235

National Aeronautics and Space Administration. (2022). NASA software engineering requirements (NPR 7150.2D, Appendix A). https://nodis3.gsfc.nasa.gov/displayDir.cfm?Internal_ID=N_PR_7150_002D_&page_name=AppendixA

Jest. (s. f.). Jest CLI options (versión 30.0). https://jestjs.io/docs/30.0/cli

Bakaus, P. (s. f.). Impeccable [Repositorio de software]. GitHub. https://github.com/pbakaus/impeccable

Kowalski, E. (s. f.). Skills for designers and engineers [Repositorio de software]. GitHub. https://github.com/emilkowalski/skills